

# Hands-On Graph Neural Networks Using Python

## Chapter 10 상세 정리

A7610 그래프신경망특론 / 조남언

June 10, 2026

### Contents

|          |                                             |          |
|----------|---------------------------------------------|----------|
| <b>1</b> | <b>개요: 링크 예측이라는 과제</b>                      | <b>2</b> |
| 1.1      | 전통적 휴리스틱                                    | 2        |
| 1.2      | 데이터 분할의 특수성                                 | 2        |
| <b>2</b> | <b>Graph Autoencoder (GAE)</b>              | <b>2</b> |
| 2.1      | 모델 구조                                       | 2        |
| 2.2      | 학습 손실                                       | 2        |
| <b>3</b> | <b>Variational Graph Autoencoder (VGAE)</b> | <b>3</b> |
| 3.1      | 확률적 인코더                                     | 3        |
| 3.2      | ELBO와 KL 정규화                                | 3        |
| 3.3      | PyG 구현 핵심                                   | 3        |
| 3.4      | Cora 실험 결과                                  | 3        |
| <b>4</b> | <b>SEAL: 서브그래프 분류로서의 링크 예측</b>              | <b>3</b> |
| 4.1      | 문제 재정의                                      | 3        |
| 4.2      | Enclosing Subgraph 추출과 타깃 누설 방지             | 4        |
| 4.3      | DRNL (Double-Radius Node Labeling)          | 4        |
| 4.4      | DGCNN                                       | 4        |
| 4.5      | Cora 실험 결과                                  | 4        |
| <b>5</b> | <b>비교와 시사점</b>                              | <b>4</b> |

## 1 개요: 링크 예측이라는 과제

링크 예측(link prediction)은 그래프  $G = (V, E)$ 에서 관측되지 않은 노드 쌍  $(u, v) \notin E$ 가 실제로 연결되어 있을(또는 미래에 연결될) 확률을 추정하는 문제다. 추천 시스템의 사용자-아이템 연결, 지식 그래프의 누락 사실 보완, 단백질-단백질 상호작용 예측 등 응용 범위가 매우 넓다.

### 1.1 전통적 휴리스틱

도메인 지식에 기반한 점수 함수들이 오랫동안 활용되었다.

- Common Neighbors:  $s(u, v) = |\Gamma(u) \cap \Gamma(v)|$
- Jaccard:  $|\Gamma(u) \cap \Gamma(v)| / |\Gamma(u) \cup \Gamma(v)|$
- Adamic-Adar:  $\sum_{w \in \Gamma(u) \cap \Gamma(v)} 1 / \log |\Gamma(w)|$
- Katz: 모든 경로 길이를 감쇠 계수로 합산

이들은 *enclosing subgraph*(두 노드를 둘러싼 작은 구조)의 함수라는 공통점이 있다. SEAL은 바로 이 점에서 출발해 GNN으로 그 함수 자체를 학습한다.

### 1.2 데이터 분할의 특수성

링크 예측은 노드 분류와 달리 엣지 단위로 train/val/test를 나눈다. PyG의 RandomLinkSplit은 그래프를 잘라낸 후 양성/음성 엣지 인덱스를 함께 제공한다. 평가 지표로는 AUC와 AP(평균 정밀도)가 표준이다.

## 2 Graph Autoencoder (GAE)

### 2.1 모델 구조

GAE는 단순한 인코더-디코더 구조다.

$$\text{인코더: } \mathbf{Z} = \text{GCN}(X, A) \in \mathbb{R}^{N \times d}$$

$$\text{디코더: } \hat{A} = \sigma(\mathbf{Z}\mathbf{Z}^\top)$$

디코더가 내적이라는 점이 핵심이다. 이는 “비슷한 노드끼리 임베딩 공간에서 가깝다”라는 단순한 유사도 가설에 기반하며, 모든 노드 쌍에 대한 점수를 행렬곱 한 번으로 얻을 수 있어 추론이 빠르다.

### 2.2 학습 손실

양성 엣지  $E^+$ 와 무작위 음성 엣지  $E^-$ 에 대한 이진 교차엔트로피:

$$\mathcal{L}_{\text{recon}} = - \sum_{(u,v) \in E^+} \log \hat{A}_{uv} - \sum_{(u,v) \in E^-} \log(1 - \hat{A}_{uv}).$$

클래스 불균형이 심하므로 양/음 같은 수로 샘플링한다. 한계는 명백하다: 임베딩이 결정적이라 모델의 불확실성을 표현할 수 없다.

### 3 Variational Graph Autoencoder (VGAE)

#### 3.1 확률적 인코더

VGAE는 잠재변수  $\mathbf{z}_i$ 를 가우시안 분포로 본다.

$$q(\mathbf{z}_i | X, A) = \mathcal{N}(\boldsymbol{\mu}_i, \text{diag}(\boldsymbol{\sigma}_i^2)), \quad \boldsymbol{\mu} = \text{GCN}_{\boldsymbol{\mu}}(X, A), \log \boldsymbol{\sigma} = \text{GCN}_{\boldsymbol{\sigma}}(X, A).$$

재매개화 트릭으로 학습 가능한 샘플을 뽑는다:  $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$ . 디코더는 GAE와 동일하다.

#### 3.2 ELBO와 KL 정규화

변분 추론의 ELBO를 최소화 형태로 쓰면:

$$\mathcal{L}_{\text{VGAE}} = \mathcal{L}_{\text{recon}} + \frac{1}{N} \text{KL}[q(Z | X, A) \| p(Z)],$$

사전분포  $p(Z) = \mathcal{N}(0, I)$ 이며 KL은 정규화 효과를 준다.  $1/N$ 은 노드 수에 비례한 KL 누적을 노드당 평균으로 환원하기 위한 가중치이며, 책의 구현에서 사용된 관용적 형태다.

#### 3.3 PyG 구현 핵심

```
from torch_geometric.nn import GCNConv, VGAE

class Encoder(torch.nn.Module):
    def __init__(self, dim_in, dim_out):
        super().__init__()
        self.conv1 = GCNConv(dim_in, 2 * dim_out)
        self.conv_mu = GCNConv(2 * dim_out, dim_out)
        self.conv_logstd = GCNConv(2 * dim_out, dim_out)
    def forward(self, x, edge_index):
        x = self.conv1(x, edge_index).relu()
        return (self.conv_mu(x, edge_index),
                self.conv_logstd(x, edge_index))

model = VGAE(Encoder(dataset.num_features, 16)).to(device)
```

VGAE 래퍼가 `recon_loss`, `kl_loss`, `test`를 모두 제공하므로 학습 코드는 매우 짧다.

#### 3.4 Cora 실험 결과

- Encoder:  $\text{GCN}(1433 \rightarrow 32) \rightarrow \text{GCN}(32 \rightarrow 16) \times 2$ .
- 300 epoch, Adam (lr=0.01).
- 결과: 테스트 AUC  $\approx 0.88$ , AP  $\approx 0.90$ .

학습이 끝나면 임의의 노드 쌍에 대한 점수를  $\hat{A} = \sigma(ZZ^\top)$ 로 한꺼번에 얻을 수 있다.

### 4 SEAL: 서브그래프 분류로서의 링크 예측

#### 4.1 문제 재정의

SEAL은 링크 예측을 이진 그래프 분류로 환원한다. 후보 링크  $(u, v)$ 의  $k$ -hop enclosing subgraph를 추출해, “이 서브그래프는 두 노드가 연결된 상태인가?”를 GNN으로 판단한다.

## 4.2 Enclosing Subgraph 추출과 타깃 누설 방지

`k_hop_subgraph`로 후보 두 노드의 2-hop 이웃 합집합을 얻은 뒤, 학습 시에 모델이 정답을 보지 않도록 타깃 엣지 자체를 서브그래프에서 제거한다.

```
sub_nodes, sub_edge_index, mapping, _ = k_hop_subgraph(
    [src, dst], 2, dataset.edge_index, relabel_nodes=True)
src, dst = mapping.tolist()
mask1 = (sub_edge_index[0] != src) | (sub_edge_index[1] != dst)
mask2 = (sub_edge_index[0] != dst) | (sub_edge_index[1] != src)
sub_edge_index = sub_edge_index[:, mask1 & mask2]
```

이 처리는 양성 샘플뿐 아니라 음성 샘플에도 동일하게 적용해야 분포가 일치한다.

## 4.3 DRNL (Double-Radius Node Labeling)

GNN은 서브그래프 안의 어떤 노드가 “타깃”인지 모른다. 따라서 거리 정보를 명시적으로 노드 라벨로 부여한다. 타깃이 아닌 노드  $x$ 에 대해  $d_x = d(x, u)$ ,  $d_y = d(x, v)$ 라 하자.  $d = (d_x + d_y)/2$ ,  $r = (d_x + d_y) \bmod 2$ 일 때 DRNL 라벨은

$$z(x) = 1 + \min(d_x, d_y) + d \cdot (d + r - 1),$$

이며 타깃 두 노드는  $z = 1$ 로 고정, 도달 불가 노드는  $z = 0$ 이다. 두 거리를 단일 정수로 인코딩하여 원-핫 표현이 깔끔하며, 거리 기반 휴리스틱들의 표현력을 포괄한다.

거리 계산의 한 가지 트릭:  $d_x$ 를 계산할 때는 그래프에서  $v$ 를 잠시 제외하고,  $d_y$ 를 계산할 때는  $u$ 를 제외한다. 그렇게 해야 두 거리가 “상대방을 거쳐가는 경로”로 줄어드는 누설을 막을 수 있다.

## 4.4 DGCNN

SEAL의 분류기는 DGCNN이다.

1. 4개의 GCN 레이어( $32 \rightarrow 32 \rightarrow 32 \rightarrow 1$ ,  $\tanh$  활성화)의 출력을 레이어별 **concat**.
2. **SortPooling**: 마지막 채널 값으로 노드를 정렬해 상위  $k = 30$ 개만 남김. 그래프 크기가 달라도 고정 길이 표현이 된다.
3. Conv1D + MaxPool로 정렬된 시퀀스를 처리(텍스트 분류와 유사).
4. Linear + Sigmoid로 링크 존재 확률 출력.

## 4.5 Cora 실험 결과

- 30 epoch, Adam ( $\text{lr}=10^{-4}$ ), BCEWithLogitsLoss.
- 결과: 테스트 AUC  $\approx 0.89$ , AP  $\approx 0.90$ . VGAE와 동급이거나 약간 우위. 표현력은 좋지만 썩마다 서브그래프 추출/전처리가 필요해 비용이 크다.

## 5 비교와 시사점

|       | VGAE                      | SEAL                      |
|-------|---------------------------|---------------------------|
| 관점    | 그래프 전체 임베딩                | 후보 링크별 서브그래프 분류           |
| 입력 단위 | 그래프 1개                    | 후보 링크 1개                  |
| 디코더   | 내적 $\sigma(z_u^\top z_v)$ | DGCNN                     |
| 노드 라벨 | 원본 feature                | 원본 feature + DRNL one-hot |
| 학습 비용 | 작음                        | 큼 (전처리 비싼)                |
| 추론 비용 | 매우 작음 ( $O(d)/\text{썩}$ ) | 큼 (썩마다 서브그래프)             |
| 표현력   | 대칭/저차원 한계                 | 휴리스틱 일반화, 풍부              |

## 언제 무엇을 쓰나

- 노드 수가 크고 후보 쌍이 많다면 VGAE처럼 한 번 학습-즉시 점수 모델이 유리.
- 적은 양이라도 정밀한 후보 평가가 필요하다면 SEAL이 우수.
- 두 접근은 결합 가능 — VGAE로 후보 풀을 거른 뒤 SEAL로 재정렬하는 식의 캐스케이드도 자연스럽다.

**이번 강의 의의** GAE/VGAE는 표현 학습의 관점, SEAL은 문제 환원의 관점이다. 같은 과제를 어떤 단위로 보느냐가 모델 구조와 비용 특성을 전혀 다르게 만든다는 점이 핵심.